SECP3133 · HIGH PERFORMANCE DATA PROCESSING

Optimizing High-Performance Data Processing for Large Scale Web Crawlers

A crawling, cleaning, and multiprocessing-optimized data pipeline built on the BERNAMA English news portal

TEAM DATADIGGERS

Nurul Ika Syafiny Binti Azhar — A23CS0164
Lubna Al Haani Binti Radzuan — A23CS0107
Nur Firzana Binti Badrus Hisham — A23CS0156
Nuraisyah Binti Mohd Zikre — A23CS0160

Lecturer: Dr. Seah Choon Sen · Section 2 · Semester 2, 2025/2026

Background & Objectives

Building a complete, evaluable data pipeline for the HPDP course

Project Background

The HPDP course requires a pipeline that collects, processes, and stores at least 100,000 structured records from an approved Malaysian website, then evaluates it with high-performance computing techniques.

We selected the BERNAMA English news portal — Malaysia's national news agency — for its structured layout and large archive of articles, then built a baseline crawler, a cleaning pipeline, and a multiprocessing-optimized version for performance comparison.

101,212

structured news records collected — exceeding the 100,000 target

Project Objectives

1. Develop an automated web crawler to collect large-scale news articles from BERNAMA
2. Extract structured fields: headline, publication date, section, summary, and URL
3. Clean the data — remove duplicates, standardize formats, validate records
4. Optimize the cleaning pipeline using multiprocessing for higher throughput
5. Evaluate baseline vs. optimized performance: time, throughput, CPU, memory

System Architecture

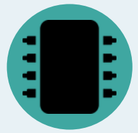
Three stages: data collection → data cleaning → optimization & evaluation

1. Data Collection

Crawl BERNAMA via sequential article IDs; extract structured fields with a multi-threaded engine.

2. Data Cleaning

Remove duplicates, standardize formats, validate URLs & dates, engineer new columns.



3. Optimization & Evaluation

Parallelize cleaning with multiprocessing; compare against baseline on 4 metrics.

Tools & Frameworks

Python | Requests + BeautifulSoup | ThreadPoolExecutor | Pandas | Multiprocessing | Psutil | Matplotlib | Google Colab / Drive

Data Collection Methodology

A predictable-ID crawl strategy, accelerated with concurrency

1. ID Page Generation

Generate sequential article URLs directly — no pagination crawling needed.

2. Concurrent Ingestion

ThreadPoolExecutor with 5 workers fetches pages simultaneously.

3. HTML Parsing

BeautifulSoup extracts headline, date, section, and summary by CSS class.

4. Progressive Append

Records stream directly to CSV to avoid holding data in memory.

5. Checkpoint Filtering

A URL hash set drops duplicates and enables safe resume.

6. Google Drive I/O

Deduplicated rows are committed to persistent cloud storage.

Dataset Volume & Ethical Crawling

Scaling to 100,000+ records responsibly

101,212

valid records scraped

~20%

yield ratio from ID sweep

0

duplicate records

5

max concurrent workers

Server Etiquette & Ethical Safeguards

User-Agent Transparency

Explicit, identifiable request headers — no spoofing.

DoS Prevention

Concurrency capped at 5 threads to protect host stability.

Resilient Backoff

Auto retry/backoff on HTTP 429 / 500 / 503 responses.

Data Cleaning Pipeline

Six sequential steps applied to the raw 101,212-record dataset

1. Duplicate removal — deduplicated by url (not headline)
2. Whitespace stripping on all text fields
3. Date standardization to YYYY-MM-DD HH:MM
4. URL validation — must start with bernama.com domain
5. Summary length filtering — drop entries under 40 characters
6. Text standardization — section field to Title Case

Records Retained at Every Stage

Raw scraped data	**101,212**
After duplicate removal	**101,212**
After date validation	**101,212**
After URL validation	**101,212**
After summary length filter	**101,212**
Final cleaned dataset	**101,212**

No records were lost — the crawler's structural quality meant every field passed validation.

Final Dataset Structure

101,212 records × 9 columns — 5 original fields + 4 engineered features

Column	Type	Description
headline	string	Article headline
publication_date	string (datetime)	Standardized publication timestamp
section	string	News category, Title Case
article_summary	string	First substantial paragraph of the article
url	string	Unique article URL
pub_date_only	date	Derived — calendar date portion
pub_year	integer	Derived — year of publication
pub_month	integer	Derived — month of publication
headline_word_count	integer	Derived — word count of headline

Derived columns turn the dataset from raw scraped text into something ready for time-based and text-based analysis.

Optimization: Multiprocessing

Restructuring the single-threaded baseline into a parallel two-phase pipeline

Phase 1 — Global Pre-Step

Deduplication by URL runs once on the full dataset, outside the parallel section, using pandas' vectorized drop_duplicates() — ensuring chunks are disjoint before splitting.

Phase 2 — Parallel Chunk Processing

The deduplicated data is split into N interleaved chunks (df.iloc[i::N]), one per CPU core, cleaned independently via multiprocessing.Pool.map(), then merged with pandas.concat().

Baseline vs. Optimized — Implementation Summary

	Baseline	Optimized
Execution model	Single process, sequential	Multiple processes, parallel
Parallelism unit	Whole dataset	Split into N chunks
Library	pandas	pandas + multiprocessing.Pool
Remove duplicate	Performed on full dataset	Performed once globally, pre-split
Merge step	Not applicable	Concatenate cleaned chunks

Challenges & Limitations

Engineering trade-offs across crawling, cleaning, and parallelization

Challenges Faced

- Headline-based dedup incorrectly dropped ~3,600 valid records — fixed by deduplicating on URL instead
- Inconsistent raw date formats — handled with errors='coerce' in pd.to_datetime()
- Tuning filters (URL, summary length) to avoid excluding valid records or admitting invalid ones
- Ensuring URL dedup happens globally before chunk-splitting, so parallel chunks stay disjoint

Limitations

- Crawler depends on BERNAMA's current HTML structure — layout changes could break extraction
- Cleaning uses basic heuristics rather than deeper content-quality validation
- Multiprocessing gains are workload-dependent — on a 2-core setup, fixed process overhead outweighed the per-record cleaning cost
- Effectiveness of parallelization should be evaluated per dataset size and core count, not assumed

Conclusion & Future Work

The project successfully built and evaluated a full crawl-clean-optimize pipeline, collecting 101,212 records from BERNAMA. The key engineering finding: for ~100K-row datasets, pandas' vectorized baseline is already so fast that multiprocessing overhead can outweigh its benefits on limited hardware — effectiveness is workload- and hardware-dependent, not automatic.

Future Work

Distributed Computing

Migrate to Apache Spark / Dask as data scales to millions of records

Incremental Processing

Delta-processing model — only feed new or modified raw records

ML-Based Validation

Replace heuristic filters with NLP models for content quality

Columnar Storage

Move from CSV to Apache Parquet for compression and faster I/O

Thank You